Jordan Pauzie, Julian Savini, Charlie Zhang
CS: 151-2 Special Topics in Data Infrastructure and Systems
Professor Lillethun & Professor Sundar
12/6/25

Project 4 Final Writeup

Executive Overview:

Picking up from where left off in the beta, we initially approached the final phase as an opportunity to implement parallelizations and refine our current codebase. However upon the discovery of a major bug that will be described in further detail in the following sections, we had to essentially start from square one again. Nonetheless, we were eventually able to implement optimizations such as Young Brothers Wait Concept (YBWC) and incorporating Jamboree Search into a parallelized Principle Variation Search (PVS).

Final Performance Optimizations:

When we regrouped after Thanksgiving break, we did a review of our entire codebase and testing methodologies firstly as a sanity check, and secondly to confirm that we were on the right track. Unfortunately we discovered a major bug in our code which had been initially masked by our testing scripts. During the beta process, our main metric for bot performance success was tracking how deep it got during searches, however this proved to be misguided. Observe this log that was output by one of our testing scripts:

Optimized (3000ms limit):
Average nodes searched: 174736
Average search time: 211ms
Max depth reached: 99
Depth progression: 99

Our testing script had a cap for how many depths it could track (99), so presumably, the "optimized" bot reached depths of far deeper than 99. In fact, after further testing, we discovered that our bot was capable of traversing to a depth of more than 7000 in roughly 3000ms. Given that and the average nodes searched shown in the log, this indicated that the bot barely interacted with any of the nodes within a particular depth and simply sped its way through. This resulted in a deep, but incredibly narrow search, and significantly handicapped our bot's abilities. Against the baseline bot, our "optimized" bot lost every single time. Eventually, we discovered that the culprit for this bug was a Zobrist key update issue, and subsequently patched it. Nonetheless, this was a major blow because it meant that we were essentially back to square one again. Additionally, our bot now had a 50% win rate against the baseline.

There are several optimizations which we added to the final. We identified enough evidence that these changes were improving our speed and/or Elo, such that we felt comfortable moving them to our final bot. One of our initial optimizations was to replace the square_eq() function call in board.c, with a direct integer comparison "cached.val != INVALID_SQUARE.val". The square_eq function is a wrapper that checks if the value of *a* is equal to the value of *b*, so we can eliminate function call overhead by directly comparing these integers. We learned that our

bot was reaching an average depth of 6, whereas our baseline bot had an average depth of 5.8 [for 3000ms searches]. Thus, we confirmed although small, this was a good optimization.

In search.c, we observed that the sort_insertion() call could be moved outside the move evaluation loop in search.c, and still provide fine results. Move sorting only needs to occur once per search node, not for each move evaluated. A lot of redundant sorting operations were being performed, and we identified, although the depth remained at 6 for 3000ms searches, the optimized bot won 55% of its matches as opposed to our previous bot winning 50% of its matches.

We leveraged parallelism to implement the Jamboree algorithm. As discussed in the class, Jamboree is a parallel search algorithm that searches multiple moves in parallel. We used Cilk's work stealing scheduler to balance the work load. It is important to note that we used a MAX_PARALLEL_MOVES=16 parameter to prevent too many threads running, creating too much overhead. During fine tuning, we set MAX_PARALLEL_MOVES to 8, as we observed we were able to reduce the time of overhead from 1800ms to 1000ms, for a 3000ms search. We also had the bot only parallelize when the depth was greater than 2, and when the number of moves per parent node was greater than 3. There is no reason to create overhead if there are only a few leaf nodes. We did not observe any improvement in Elo for matches in which the bot was performing 1000ms searches in game. However, we did observe that it won 6/6 games tested with 250ms searches in game. At this time, we had no idea what was the mean search time for a bot which had no set time limit on searches, so it was hard to identify if a higher win rate for smaller searches was significant. But, since our baseline bot was naturally winning half of its games, we decided that this, on top of the improvement with overhead, suggested this optimization was worth implementing.

While implementing our Jamboree algorithm, we were able to implement a serialized YBWC search strategy. Upon the completion of the Jamboree algorithm, we decided to integrate the two optimizations and design a parallelized PVS that leveraged YBWC and Jamboree search. In this combined approach, YBWC ensures that the most promising move, denoted as the "youngest brother", is searched first serially, increasing the likelihood of an early beta cutoff. Subsequent moves, or "older brothers," are then evaluated in parallel, taking advantage of speculative parallelism while minimizing wasted computation on subtrees that are likely to be pruned. By signaling parallel tasks to abort immediately upon a beta cutoff, we reduced unnecessary work and improved overall efficiency. With this approach we were able to achieve a win rate of 55%. Interestingly enough, the average time per test for our serialized YBWC was 3000 ms in contrast to 4200 ms for our parallelized version.

With all of the optimizations above implemented, we decided to try adding some more granularity to our search process to hopefully preserve computational efficiency but improve end game performance. Thus we tried extending the search depth by 1 whenever the monarch was being threatened by the opposition monarch's laser. This would allow the search to analyze the situation of its pieces and that of its opposition pieces more deeply. With this granular

optimization, we were able to achieve a 60% win rate against the baseline and reach a depth of 8 after 30 seconds versus 7 for the baseline.

Failed Performance Optimizations:

One failed optimization was the narrowing of alpha-beta windows. Instead of initializing the search with the full window at the root (-INF to INF), we utilized a narrow window around the score returned by the previous iteration of iterative deepening. For instance if depth 1 finds a score of +150, we would then for depth 2, add and subtract 100 to get our updated upper and lower bounds [alpha and beta]. We hypothesized that the true score would likely remain close to the previous score. By narrowing the window we expected to induce more cutoffs, and therefore when comparing the bot to our baseline, it should be able to reach a further depth in the same amount of time. But this optimization proved inefficient. Our hypothesis was false, because if the real optimal move was +300, you would still have to re-search regardless, and this would only work if the first depth was representative of an optimal search. During the test, the optimized bot scored the same elo for 20 games, and depth did not improve during a 1 second or three second search. We removed this optimization. We also tried implementing aggressive alpha-beta pruning at the start of the search, and then widen the bounds as this game continued to play out. Unfortunately, we saw no improvement in Elo, which was surprising. In truth, we would need to fine tune these rates so that the bot could appropriately scan more moves, but we didn't have the time to do so.

Another optimization we tried implementing was modifying the transposition table (TT) to prefer replacing older entries of the same depth. The rationale was that older entries of the same depth were less likely to be relevant to the current search. However our win rate actually suffered as a result, dipping slightly below a 50% win rate. In the end, this proved to be just another step into the careful fine tuning we needed to apply in order to squeeze performance out of the TT.

We also thought it'd be interesting if we could improve our bot's tactical play by placing more emphasis on recapture scenarios. With this in mind, we tried adding a search extension when recapturing on the same square an opponent just moved to. However, once again, our win rate against the baseline ended up lower than 50%. Reflecting critically, we believe that the much of the recapture scenarios we were targeting had already been covered by our already-implemented capture extension. Furthermore, extending recaptures may have extended too many positions, reducing effective search depth elsewhere, and triggering false positives.

Another failed optimization was fine tuning the decay factor. With very little time left to complete the final code, and seeing not much improvement in depth/sec, we decided to adjust the decay factor from 0.9 to 0.92. The history scores now retain 92% of their value each update instead of 90%. Although this looks to be a minor change, the impact can be very significant. If 10 updates are performed, a 0.9 decay rate means .9 ^ 10 = ~35% of the original scores would remain. With a 0.92 decay rate, ~44% of the original scores would remain. Essentially, this means that we are making the engine "forget slower". In chess variants like Leiserchess, a move that is good in

one position is often good in many nearby positions in the search tree. By keeping the score higher for longer, the engine remembers that this move is strong and will try it earlier in future searches. But with higher value of the factor, the engine will keep trying moves that were good long ago but might be bad now. We saw a major decrease in win rate, and the same occurred when we tried decreasing the decay factor to 0.88.

We also tried multiplying the history scores by 1.1x when using them for move ordering. The history scores are used when creating sortable moves. The idea was that after looking up the history score from the best_move_history table, we would multiply it by 1.1x before it is used in the move ordering. This would affect the key field in the sortable_move_t struct, so that good history moves would be ranked higher, as compared to other move ordering factors like killers and captures. Prioritizing moves that worked well in similar positions showed no improved win rate against the baseline, so we decided to not implement this.

We also tried a last minute attempt at Late Move Reduction. We decided to add more reductions beyond the traditional 1-2 ply reduction. Whereas with traditional LMR logic, you would remove 1 ply of depth after searching LMR_R1 moves, and then 2 plies after searching LMR_R2 moves, we added an additional ply reduction after depths of 3, and an additional reduction for any depth past 5. And a safety cap was implemented to ensure reductions never exceeded the depth.

Identification of Bottlenecks [Post Beta]:
With the conclusion of our beta code, we decided to perform a profiling report for each individual ./leiserchess command akin to the one we performed for the design document. The commands we tested were display, eval, generate, status, go, move and perft. After investigating the profiling data for each command, we discovered that the following functions monopolized the CPU:

- The get_piece_at() function was, before optimization, 21.2% of compute for running perft. Now, after optimization, get_piece_at() only is 6.18% of compute for running perft. Inlining was very effective.
- Originally, square_to_index() was taking 7.5% of compute for running perft. Now, after inlining, square_to_index() is only taking 1.78% of compute for running perft.

As we continue to reduce large bottlenecks, there are still bottlenecks [much smaller] that still arise. Here are a couple of functions which are current bottlenecks. In the “Ideals for Final Submission” section, we outline how we can perhaps improve these functions.

- generate_all() [12.73% cpu for perft]
- recompute_zob() [10.21% cpu for perft]
- toggle_piece() [9.75% cpu for perft]

Post Final Retrospective:
Our team did a great job re-evaluating, immediately after beta, how we wanted to test our code. We used nodes per second to look at the performance of our code, and switched to depth per *t* time, to see how many moves ahead our bot is analyzing. Once we realized our bot was searching depths greater than 7,000 in three seconds, we became skeptical of our optimizations. In reality, once you are performing alpha-beta search past a depth of 20, the

performance will very likely plateau. Additionally, we needed to add shell scripts to our codebase, to quickly simulate games to get an Elo rating, and continue to analyze depth per t seconds. There were several limitations we faced along the way. Until we learned our bot was frequently performing moves every 600ms, we were using arbitrary times for testing our depth and Elo, and therefore it was very tricky to identify for how long we should be testing depths and games. We wanted to select times that would be favorable for our bot, but we also wanted our bot to perform well under a variety of constraints. In sum, setting testing parameters was one of the more challenging aspects of this project.

After receiving permission to test code locally, we could now perform quick tests, and not have to make hundreds of commits to then run our code on Stampede3. Originally, we compared speed up percentage, which can at least help a little when running leiserchess across multiple systems. However, purely testing depth and Elo can vary across systems, and as we were implementing optimizations, it was hard for us to know how well optimizations performed unless we tested code on our three laptops, which didn't really happen. Furthermore, testing Elo for 10 games took us ~8 minutes, so it was time consuming to see the effectiveness of our changes.

Lastly, our team is happy we tried a variety of optimizations, even though we didn't see much improvement in depth. We were surprised that parallelization was providing no speedup, and even running versions where we only parallelized the first couple of layers, still was presumably producing enough overhead to slow our system down. Rather than giving up, we still applied parallelized alpha beta search w/ jamboree search, young brothers wait, and more to at least practice understanding these algorithms and applying them in practice, which was very beneficial for our learning.

Performance Progress [Beta]:
Original bot vs. optimized bot :

Version	**Absolute Speedup**	**Changes (if any)**
Baseline	0%	
Version 1	+34%	Inlining get_piece_at()
Version 2	+62%	Inlining square_to_index()
Version 3	+74%	Implemented monarch caching
Version 3	+74%	Reducing extraneous square_to_index call
Version 5	+77%	Replaced insertion sort with merge sort

Performance Progress [Final]:
We only add the changes which improved our results to this table.

Version	Optimization	Avg Depth (3 sec)	Win Rate vs Baseline (20 games)	Overhead (to run 3 sec search)
Beta	N/A	5.8	50%	N/A
Version 1	Replaced square_eq() with direct comparison	6.0	50%	N/A
V2	Moved sort_insertion() outside loop	6.0	55%	N/A
V3	Jamboree Parallelization	6.0	55%	1300ms
V4	Young Brothers Wait	6.0	55%	1200ms
V5	Monarch Laser Threat Extension	6.0	60%	1200ms

Team Dynamics/Division of Work [Beta]:
Remaining consistent with our design document, our team did a great job prioritizing the project throughout the week, and the three of us were pretty consistently on call together, testing new ideas and debugging. Generally, we were open to an idea proposed in the project 4 guideline, saying each section should have a “CEO” throughout the project. Generally speaking, Charlie had ownership over the mergesort algorithm. Julian had ownership over the monarch position caching, and scripting to test depth %. Jordan had ownership over testing and attacking performance bottlenecks with inlining. But, we all spent time helping each other debug and test these features. Overall, we are happy with our beta submission, as we were able to comfortably do a congruent depth search in less than half the time.

Team Dynamics/Division of Work [Final]:
On December 1 and 2, Julian worked on a couple of optimizations (V1 and V2), and the parallelization of alpha beta search. After Julian and Charlie met on Wednesday, and investigated the code to realize that the aggressive depth searching was causing the bot to perform poorly against the baseline bot. Julian added shell script files for testing (with assistance from an AI agent), to assess depth during a set time, elo given a set depth, and elo given a set time per move. Julian then spent Wednesday night reapplying the optimizations to pinpoint which optimization was causing aggressive depth searching, and thus poor elo. After he finally identified the issue on Thursday, he tried parallelizing alpha beta search again. On Friday, the team collectively spent the afternoon trying to debug/reimplement the alpha beta search with Jamboree, and implementing Young Brothers Wait. During this time, Julian used an AI agent to test aggressive pruning. Then, for the evening and night, Julian tried a variety of optimizations, from recapture extensions, increased decay rate and history scoring, and depth

dependent LMR, using AI to help test. Furthermore, he was able to use the monarch laser threat extension optimization to reach a 60% win rate.

Project Log:

#	Date	Duration	Description
1	Fri, Nov. 14	1	Formed team and created team contract
2	Tue, Nov. 18	3	Started the design document and began completing checkoff
3	Wed, Nov. 19	2	Finished design document and checkoff
4	Sat, Nov. 22	2	Implemented optimizations such as function inlining, monarch caching, merge sort
5	Sat, Nov. 22	2	Debugging illegal move issue which stemmed from one of our optimizations
6	Sun, Nov. 23	3	Solved the illegal move bug by removing a piece of improperly optimized code
7	Mon, Nov. 24	2	Added script to test suite; see percentage of improvement
8	Mon, Nov. 24	2	Completed code for beta and significantly improved absolute performance in terms of speed
9	Mon, Nov. 24	2	Beta Writeup
–	FINAL	–	–
10	Mon, Dec. 1	1	Testing various inlining, using depth/sec as core metric [with zobrist bug]
11	Mon, Dec. 1	1	Experimentation w/ moving insertions sorts out of core loop logic, still using depth/sec as core metric [with zobrist bug]
12	Tue, Dec. 2	2	Re-evaluated how we are testing speed of code and updated testing scripts + initial attempt at parallelizing alpha beta search
13	Wed, Dec. 3	1.5	After attending office hours, write test scripts to test Elo for set depth and set time. Realized “optimized bot”
14	Wed, Dec. 3	2	Created baseline branch to strip our optimizations and identify what “optimization” was causing low Elo score.
15	Thursday, Dec. 4	2	Identified a zobrist key update failure. Pushed to baseline and will add optimizations from there

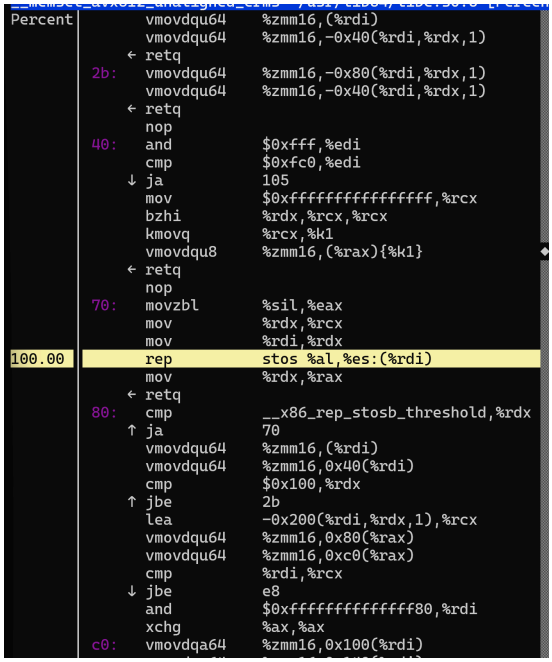
16	Thursday, Dec. 4	1.5	Replaced square_eq() function with a direct integer comparison in get_monarch() + moved sort_insertion() outside of for loop
17	Friday, Dec. 5	1	Drafted list of potential optimizations, + tested TT table records_per_set to be 4 instead of 1 [failed optimization]
18	Friday, Dec. 5	2	Parallelizing alpha beta search w/ jamboree
19	Friday, Dec. 5	1.5	Young brothers wait algorithm
20	Friday, Dec. 5	1	Aggressive pruning at start of game [failed optimization]
21	Friday, Dec. 5	1	Aspiration Windows Optimization [failed optimization]
22	Friday, Dec. 5	0.75	Parallelization in PV search
23	Friday, Dec. 5	1	Recapture Extension [failed optimization]
24	Friday, Dec. 5	1	Fine tuning depth decay rate and history score multiplier
25	Friday, Dec. 5	1	Extends search depth by 1 when monarch is adjacent to opponent's laser
26	Friday, Dec. 5	1	Depth Dependent LMR [failed optimization]
27	Friday, Dec. 6 [took late extension to complete write up]	2.5	Final Write Up

Additional Information & Acknowledgements:
No additional information needed.

Appendix 1 (Profiling Data Beta):
PERFT:

```
Samples: 62K of event 'cycles:Pu', Event count (approx.): 46645308844
Overhead  Command      Shared Object          Symbol
  12.73%  leiserchess  leiserchess            [.] generate_all
  10.21%  leiserchess  leiserchess            [.] recompute_zob
  10.09%  leiserchess  leiserchess            [.] square_of
   9.75%  leiserchess  leiserchess            [.] toggle_piece
   7.67%  leiserchess  leiserchess            [.] get_piece_at
   6.80%  leiserchess  leiserchess            [.] piece_index
   6.71%  leiserchess  leiserchess            [.] check_square
   6.18%  leiserchess  leiserchess            [.] get_piece_at
   5.70%  leiserchess  leiserchess            [.] make_sortable
   4.10%  leiserchess  leiserchess            [.] move_of
   2.86%  leiserchess  leiserchess            [.] player_wins
   2.13%  leiserchess  leiserchess            [.] take_cardinal_step
   2.10%  leiserchess  leiserchess            [.] get_piece_at
   1.78%  leiserchess  leiserchess            [.] square_to_index
   1.54%  leiserchess  leiserchess            [.] take_ordinal_step
   1.31%  leiserchess  leiserchess            [.] get_file_offset
   1.12%  leiserchess  leiserchess            [.] ordinal_dir_to_index
   1.02%  leiserchess  libc.so.6              [.] __memset_avx512_unaligned_erms
   0.97%  leiserchess  leiserchess            [.] get_rank_offset
   0.88%  leiserchess  libc.so.6              [.] __memmove_avx512_unaligned_erms
   0.58%  leiserchess  leiserchess            [.] make_ordinal
   0.53%  leiserchess  leiserchess            [.] square_eq
   0.34%  leiserchess  leiserchess            [.] set_piece_at
   0.30%  leiserchess  leiserchess            [.] move_piece
   0.29%  leiserchess  leiserchess            [.] step_laser
   0.24%  leiserchess  leiserchess            [.] fire_laser
   0.23%  leiserchess  leiserchess            [.] cardinal_of_file_dir
   0.20%  leiserchess  leiserchess            [.] cardinal_of_rank_dir
   0.16%  leiserchess  leiserchess            [.] cardinal_dir_to_index
   0.15%  leiserchess  leiserchess            [.] execute_turn
   0.14%  leiserchess  leiserchess            [.] perft_search
   0.14%  leiserchess  leiserchess            [.] get_laser_origin
   0.14%  leiserchess  leiserchess            [.] get_monarch
   0.11%  leiserchess  leiserchess            [.] check_board_zob
   0.08%  leiserchess  leiserchess            [.] move_eq
   0.08%  leiserchess  leiserchess            [.] toggle_color_to_move
   0.07%  leiserchess  leiserchess            [.] get_move
   0.07%  leiserchess  leiserchess            [.] check_valid_square
   0.06%  leiserchess  leiserchess            [.] is_game_over
   0.06%  leiserchess  leiserchess            [.] color_to_move_of
   0.06%  leiserchess  leiserchess            [.] rotate_ordinal
   0.05%  leiserchess  leiserchess            [.] swap_color_to_move
   0.04%  leiserchess  leiserchess            [.] rotate_file_right
   0.04%  leiserchess  leiserchess            [.] rotate_piece
   0.03%  leiserchess  leiserchess            [.] opp_color
   0.03%  leiserchess  leiserchess            [.] rotate_rank_right
   0.03%  leiserchess  leiserchess            [.] rotate_cardinal
   0.02%  leiserchess  [unknown]              [k] 0xffffffff8b601122
   0.02%  leiserchess  leiserchess            [.] fil_of
   0.02%  leiserchess  leiserchess            [.] rnk_of
   0.01%  leiserchess  leiserchess            [.] memcpy@plt
   0.01%  leiserchess  leiserchess            [.] cardinal_dir_eq
   0.00%  leiserchess  leiserchess            [.] update_victim
   0.00%  leiserchess  libc.so.6              [.] __vfprintf_internal
   0.00%  leiserchess  libxalt_init.so        [.] __XALT_host_switch_to_buffer
   0.00%  leiserchess  ld-linux-x86-64.so.2   [.] intel_check_word.constprop.0
   0.00%  leiserchess  ld-linux-x86-64.so.2   [.] __GI___tunables_init
   0.00%  leiserchess  ld-linux-x86-64.so.2   [.] _dl_start
Tip: Create an archive with symtabs to analyse on other machine: perf archive
```

ANNOTATING MEMSET:



DISPLAY:

```
Samples: 2K of event 'cycles:Pu', Event count (approx.): 488343856
Overhead  Command      Shared Object          Symbol
  97.43%  leiserchess  libc.so.6             [.] __memset_avx512_unaligned_erms
   2.21%  leiserchess  [unknown]             [k] 0xffffffff8b601122
   0.11%  leiserchess  ld-linux-x86-64.so.2  [.] _dl_check_map_versions
   0.10%  leiserchess  leiserchess           [.] init_zob
   0.04%  leiserchess  leiserchess           [.] init_board
   0.04%  leiserchess  leiserchess           [.] strcmp@plt
   0.04%  leiserchess  ld-linux-x86-64.so.2  [.] init_cpu_features.constprop.0
   0.01%  leiserchess  ld-linux-x86-64.so.2  [.] _dl_sysdep_parse_arguments
   0.00%  leiserchess  libc.so.6             [.] _IO_getline_info
   0.00%  leiserchess  ld-linux-x86-64.so.2  [.] _dl_start
   0.00%  leiserchess  libc.so.6             [.] _IO_default_uflow
   0.00%  leiserchess  ld-linux-x86-64.so.2  [.] _start
   0.00%  leiserchess  libc.so.6             [.] _IO_file_underflow@@GLIBC_2.2.5
   0.00%  leiserchess  libc.so.6             [.] read
```

GO:

```
Samples: 2K of event 'cycles:Pu', Event count (approx.): 527741834
Overhead  Command      Shared Object          Symbol
  92.64%  leiserchess  libc.so.6             [.] __memset_avx512_unaligned_erms
   1.92%  leiserchess  [unknown]             [k] 0xffffffff8b601122
   1.45%  leiserchess  leiserchess           [.] toggle_piece
   0.92%  leiserchess  leiserchess           [.] recompute_zob
   0.70%  leiserchess  leiserchess           [.] get_piece_at
   0.54%  leiserchess  leiserchess           [.] check_square
   0.43%  leiserchess  leiserchess           [.] piece_index
   0.38%  leiserchess  leiserchess           [.] square_of
   0.25%  leiserchess  leiserchess           [.] square_to_index
   0.14%  leiserchess  libc.so.6             [.] __memmove_avx512_unaligned_erm
   0.14%  leiserchess  leiserchess           [.] move_piece
   0.12%  leiserchess  leiserchess           [.] mcede_pair
   0.11%  leiserchess  ld-linux-x86-64.so.2  [.] _dl_lookup_direct
   0.07%  leiserchess  ld-linux-x86-64.so.2  [.] intel_check_word.constprop.0
   0.06%  leiserchess  leiserchess           [.] ordinal_dir_to_index
   0.05%  leiserchess  libc.so.6             [.] munmap_chunk
   0.05%  leiserchess  leiserchess           [.] generate_all
   0.02%  leiserchess  ld-linux-x86-64.so.2  [.] __GI___tunables_init
   0.00%  leiserchess  libc.so.6             [.] _IO_getline_info
   0.00%  leiserchess  ld-linux-x86-64.so.2  [.] _dl_start
   0.00%  leiserchess  libc.so.6             [.] _IO_default_uflow
   0.00%  leiserchess  libc.so.6             [.] _IO_file_underflow@@GLIBC_2.2.
   0.00%  leiserchess  ld-linux-x86-64.so.2  [.] _start
   0.00%  leiserchess  libc.so.6             [.] read
```

EVAL:

```
Samples: 2K of event 'cycles:Pu', Event count (approx.): 495096505
Overhead  Command      Shared Object          Symbol
  96.99%  leiserchess  libc.so.6              [.] __memset_avx512_unaligned_erms
   2.42%  leiserchess  [unknown]              [k] 0xffffffff8b601122
   0.26%  leiserchess  leiserchess            [.] check_square
   0.14%  leiserchess  libxalt_init.so        [.] __XALT_xmalloc_xalt_1_5
   0.09%  leiserchess  ld-linux-x86-64.so.2   [.] handle_intel.constprop.0
   0.07%  leiserchess  ld-linux-x86-64.so.2   [.] do_lookup_x
   0.02%  leiserchess  ld-linux-x86-64.so.2   [.] __GI___tunables_init
   0.01%  leiserchess  ld-linux-x86-64.so.2   [.] _dl_start
   0.00%  leiserchess  libc.so.6              [.] _IO_getline_info
   0.00%  leiserchess  libc.so.6              [.] _IO_default_uflow
   0.00%  leiserchess  ld-linux-x86-64.so.2   [.] _start
   0.00%  leiserchess  libc.so.6              [.] _IO_file_underflow@@GLIBC_2.2.
   0.00%  leiserchess  libc.so.6              [.] read
```

GENERATE:

```
Samples: 2K of event 'cycles:Pu', Event count (approx.): 488201134
Overhead  Command      Shared Object          Symbol
  97.76%  leiserchess  libc.so.6              [.] __memset_avx512_unaligned_erms
   1.94%  leiserchess  [unknown]              [k] 0xffffffff8b601122
   0.15%  leiserchess  leiserchess            [.] myrand
   0.09%  leiserchess  ld-linux-x86-64.so.2   [.] intel_check_word.constprop.0
   0.04%  leiserchess  ld-linux-x86-64.so.2   [.] check_match
   0.02%  leiserchess  ld-linux-x86-64.so.2   [.] __GI___tunables_init
   0.00%  leiserchess  ld-linux-x86-64.so.2   [.] _dl_start
   0.00%  leiserchess  ld-linux-x86-64.so.2   [.] _start
```

MOVE A3B4:

```
Samples: 2K of event 'cycles:Pu', Event count (approx.): 488549010
Overhead  Command      Shared Object          Symbol
  97.22%  leiserchess  libc.so.6              [.] __memset_avx512_unaligned_erms
   2.40%  leiserchess  [unknown]              [k] 0xffffffff8b601122
   0.11%  leiserchess  ld-linux-x86-64.so.2   [.] _dl_relocate_object
   0.11%  leiserchess  leiserchess            [.] myrand
   0.06%  leiserchess  ld-linux-x86-64.so.2   [.] update_active.constprop.0
   0.04%  leiserchess  libc.so.6              [.] buffered_vfprintf
   0.04%  leiserchess  leiserchess            [.] toggle_piece
   0.01%  leiserchess  ld-linux-x86-64.so.2   [.] _dl_sysdep_parse_arguments
   0.00%  leiserchess  ld-linux-x86-64.so.2   [.] _dl_start
   0.00%  leiserchess  ld-linux-x86-64.so.2   [.] _start
```

STATUS:

```
Samples: 2K of event 'cycles:Pu', Event count (approx.): 503503946
Overhead  Command      Shared Object          Symbol
  97.65%  leiserchess  libc.so.6              [.] __memset_avx512_unaligned_erms
   1.99%  leiserchess  [unknown]              [k] 0xffffffff8b601122
   0.13%  leiserchess  libc.so.6              [.] _int_malloc
   0.10%  leiserchess  libc.so.6              [.] getenv
   0.08%  leiserchess  ld-linux-x86-64.so.2   [.] intel_check_word.constprop.0
   0.02%  leiserchess  libc.so.6              [.] _IO_default_uflow
   0.02%  leiserchess  ld-linux-x86-64.so.2   [.] __GI___tunables_init
   0.00%  leiserchess  libc.so.6              [.] _IO_getline_info
   0.00%  leiserchess  ld-linux-x86-64.so.2   [.] _dl_start
   0.00%  leiserchess  libc.so.6              [.] _IO_file_underflow@@GLIBC_2.2.
   0.00%  leiserchess  ld-linux-x86-64.so.2   [.] _start
   0.00%  leiserchess  libc.so.6              [.] read
```